Implementação de um Sistema de Recomendação

1. Introdução

Este projeto tem como objetivo principal a **implementação de um sistema de recomendação** para clientes de lojas de varejo. Utilizando uma metodologia ativa, os alunos da disciplina de Estrutura de Dados serão guiados através de uma série de atividades práticas que culminarão na construção de um sistema funcional. A abordagem visa não apenas o desenvolvimento de habilidades técnicas em programação, mas também a compreensão do conceito teórico por trás dos sistemas de recomendação.

2. Contexto do Projeto

Sistemas de recomendação são ferramentas essenciais no cenário atual do comércio eletrônico, auxiliando clientes a descobrir produtos que possam ser de seu interesse e, consequentemente, aumentando as vendas e a satisfação do consumidor. Eles funcionam analisando o comportamento de compra e as preferências dos usuários para sugerir itens relevantes.

2.1 Método da Vizinhança

Um dos métodos mais tradicionais e intuitivos para a construção de sistemas de recomendação é o Método da Vizinhança, que se baseia na ideia de que usuários com gostos ou comportamentos semelhantes tendem a gostar dos mesmos produtos. Este método pode ser dividido em duas abordagens principais: filtragem colaborativa baseada em usuário (*User-Based Collaborative Filtering*) e filtragem colaborativa baseada em item (*Item-Based Collaborative Filtering*). Neste projeto, focaremos na abordagem baseada em usuário.

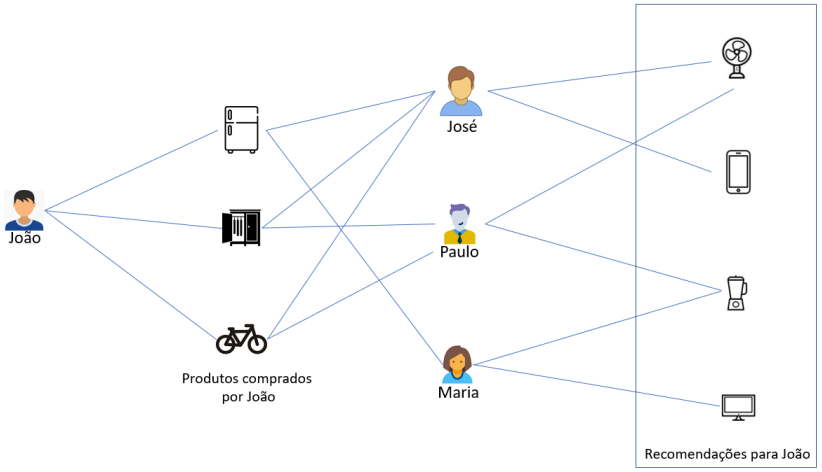

Figura 1. Ilustração do método da vizinhança. Recomendação de produtos para o cliente João baseada no histórico de compras de outros clientes.

Para aplicar o Método da Vizinhança, é fundamental representar as interações entre clientes e produtos. Isso é feito através de uma matriz de compras, onde as linhas representam os

clientes e as colunas representam os produtos. Uma entrada na matriz indica se um cliente comprou um determinado produto. A partir desta matriz, calculamos a similaridade entre clientes.

A distância de Jaccard será utilizada para medir a similaridade entre os clientes. Esta métrica é particularmente útil para conjuntos binários (como o histórico de compras, onde um produto foi comprado ou não) e é definida como o tamanho da interseção dividido pelo tamanho do conjunto de itens comprados por um dos clientes. Matematicamente, para dois clientes i e j, a similaridade de Jaccard é definida como:

$$s(i,j) = 1 - \frac{|P_i \cap P_j|}{|P_i|}. \tag{1}$$

Onde P_i e P_j são os conjuntos de produtos comprados pelos clientes i e j, respectivamente.

A Figura 2 ilustra um exemplo de relação das compras dos clientes João e Paulo. João comprou 3 produtos e Paulo comprou 4 produtos. Na interseção de produtos comprados temos dois produtos, logo, $|P_{joão} \cap P_{paulo}| = 2$.

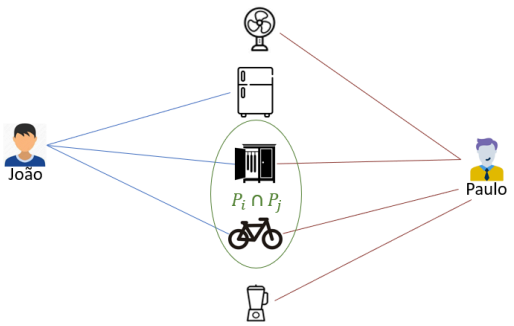

Figura 2. Relação de compras dos clientes João e Paulo e sua interseção.

Utilizando a equação (1), temos que $s(joão, paulo) = 1 - \frac{2}{3} = \frac{1}{3}$, enquanto $s(paulo, joão) = 1 - \frac{2}{4} = \frac{1}{2}$. Isso mostra que a matriz de similaridade é **assimétrica**, o que é desejável, pois evita que a presença de um grande comprador faça com que os demais sejam considerados muito semelhantes a ele, o que não refletiria a realidade.

2.2 Construção da Matriz de Similaridade

Considerando n como o número de clientes e m como o número de produtos, a matriz de compras $A_{n\times m}$ pode ser utilizada para construir a matriz de similaridade $S_{n\times n}$. A construção da matriz de similaridade passa pela construção da matriz de interseção $I_{n\times n}$ a partir da multiplicação da matriz de compras que é dada por:

$$I_{n\times n} = A_{n\times m} \times A^T_{m\times n}.$$

Onde $A^T_{m \times n}$ é a transposta da matriz de compras. Esta operação resulta em uma matriz onde cada elemento $I_{i,j}$ representa a interseção $P_i \cap P_j$ (ou o número de produtos em comum) entre o cliente i e o cliente j.

A Figura 3 ilustra a multiplicação do vetor de compras do cliente João pelos vetores de compras dos demais clientes da base, resultando no vetor de interseção das compras de João. Por exemplo, o produto escalar entre o vetor de compras de João e o de Maria é igual a 2, indicando que ambos os clientes adquiriram dois produtos em comum, especificamente os produtos B e F.

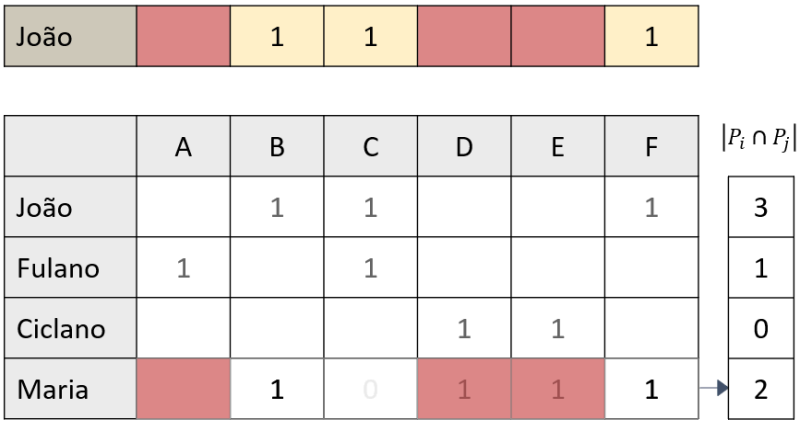

Figura 3. Computação do vetor de interseção de compras do cliente João ($I_{joão}$) com os outros clientes da base.

Por fim, para construir a matriz de similaridade $S_{n \times n}$ final, computamos cada célula da matriz como:

$$S_{i,j} \leftarrow 1 - \frac{I_{i,j}}{|P_i|} \ .$$

2.3 Algoritmo de Recomendação

O algoritmo de recomendação de produtos aos clientes se dá através do ranqueamento dos produtos com base na matriz de similaridade entre clientes. Para recomendar k produtos a um cliente c, o algoritmo segue os seguintes passos:

1. **Identificação de Vizinhos:** Crie uma lista L com todos os clientes que possuem similaridade com c (similaridade menor que 1 e que não seja o próprio cliente c);
2. **Inicialização do Vetor de Ranqueamento:** Para cada cliente, crie um vetor R de ranqueamento para todos os produtos da loja, atribuindo um valor inicial de 1 para cada produto;
3. **Cálculo do Ranqueamento:** Para cada cliente $s \in L$ e para cada produto p comprado por s e não comprado por c, atualize o ranqueamento específico do produto p fazendo: $R_p \leftarrow R_p \times s(c, s)$;
4. **Ordenação e Seleção:** Ordene o vetor R de forma não decrescente e retorne os k primeiros produtos da lista R. Estes serão os produtos melhor ranqueados e, portanto, recomendados ao cliente c.

3. Atividades do Projeto

Cada atividade é projetada para ser um módulo independente, mas interconectado, que contribui para a construção do sistema de recomendação completo. Cada atividade incluirá:

- ➔ **Descrição da Atividade:** Detalhamento do que precisa ser implementado.
- ➔ **Teoria Necessária:** Conceitos e algoritmos fundamentais para a implementação.
- ➔ **Entregáveis:** O que os alunos devem apresentar ao final da atividade.

Avaliação

A avaliação será contínua, baseada na entrega e funcionalidade de cada módulo, na clareza do código, na documentação interna e na capacidade de demonstrar a compreensão dos conceitos teóricos e práticos envolvidos. A performance e a otimização das implementações também serão critérios importantes, especialmente nas atividades que envolvem comparação de algoritmos e tratamento de matrizes esparsas.

Cronograma

Atividade	Data apresentação
1	11/03/2026
2	18/03/2026
3	25/03/2026

Cada grupo pode ser formado de duas pessoas. Posterei em um momento futuro, na sala virtual da disicplina, uma planilha para que cada grupo possa agendar seu horário de apresentação. As apresentações ocorrerão na quarta-feira em um dia diferente das aulas da disciplina que ocorrerão normalmente com o prof. substituto Gabriel Belarmino.

Atividade 1: Módulo ListaCompras

Descrição:

Implementação do módulo ListaCompras, responsável por organizar e armazenar o histórico de compras dos clientes. Será fornecido um histórico de compras como entrada, e o módulo deverá processá-lo para criar e preencher as seguintes estruturas de dados:

- ➔ Um **vetor** contendo os códigos de todos os clientes.
- ➔ Um **mapa** que associa o código original do cliente (da base da loja) a um índice interno único.
- ➔ Um **vetor** contendo os nomes de todos os produtos.
- ➔ Um **mapa** que associa o código original do produto (da base da loja) a um índice interno único do vetor de produtos.
- ➔ Um **vetor** com dimensão igual ao número de clientes, onde cada célula contém uma lista de identificadores internos dos produtos comprados pelo respectivo cliente.

A base de dados utilizada é um arquivo texto no formato CSV, que possui uma estrutura tabular, como ilustrado no exemplo a seguir:

DATA_COMPRA	COD_CLIENTE	COD_PRODUTO	NOME_PRODUTO
20200515	05090301	025728	SMARTPHONE SAMSUNG GAL J4 CORE 16GB PRETO
20200515	05190001	025172	SMARTPHONE LG K9 TV LM-X210BMW PRETO
20200515	99DL9N01	026243	SUPORTE UNIV. BEDIN BU-25
20200515	05992701	026574	SMARTPHONE SAMSUNG GAL A30S 64GB BRANCO
20200515	37578201	025367	TABLET NB729 MINI MS40G BRANCO
20200515	23210301	025728	SMARTPHONE SAMSUNG GAL J4 CORE 16GB PRETO
20200515	93168801	025354	TELEVISOR SMART 32" UN32J4290 SAMSUNG
20200515	77254401	022620	CELULAR RIU-I R200 DB BRANCO-ROSA
20200515	62982901	026584	SMARTPHONE MOTOROLA MOTO G8 PLUS 64GB CEREJA
20200515	61397001	021854	CELULAR BLU JENNY 2.8 T276 DB PRETO-AZUL
20200515	54509401	021647	DVD D-15 KARAOKE MONDIAL
20200515	95783001	025246	PEN DRIVE PD590 64GB TWIST PRETO
20200515	64669701	025996	TELEVISOR 32" 32S6500S HD LED SEMP TCL
20200515	99EOOG01	025172	SMARTPHONE LG K9 TV LM-X210BMW PRETO
20200515	99DQI601	025172	SMARTPHONE LG K9 TV LM-X210BMW PRETO

Cada linha representa o registro de uma compra individual e contém as seguintes colunas descritivas:

- ➔ **DATA_COMPRA:** data em que a compra foi realizada, no formato *aaaammdd*;
- ➔ **COD_CLIENTE:** código único que identifica o cliente no sistema de informação da loja;
- ➔ **COD_PRODUTO:** código único que identifica o produto no sistema de informação da loja;
- ➔ **NOME_PRODUTO:** nome descritivo do produto adquirido.

O preenchimento das estruturas ocorre em duas fases, conforme ilustrado na Figura 4.

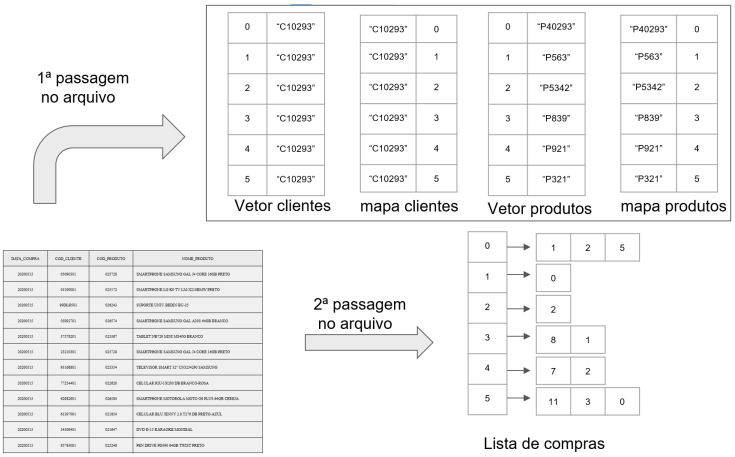

Figura 4. Esquema de leitura da base de compras e preenchimento das estruturas do módulo ListaCompras.

Na primeira fase, os códigos originais de cada cliente e produto são associados aos índices internos de controle por meio de vetores que armazenam esses códigos e de mapas reversos, os quais indicam, dado um código original, qual é o identificador interno correspondente.

Na segunda fase, é construída a lista de compras, composta por um vetor cuja dimensão corresponde ao número de clientes da base. Cada posição desse vetor contém uma lista dos índices internos dos produtos adquiridos pelo respectivo cliente.

Teoria aplicada:

<u>Leitura de Arquivos CSV</u>

Arquivos CSV (*Comma-Separated Values*) são arquivos de texto por linhas, e cada linha contém um conjunto de valores separados por vírgula.

Por exemplo, o arquivo chamado "dados.csv" pode conter o seguinte conteúdo:

```
Nome,Idade,Cidade

Ana,25,Recife

Bruno,30,Curitiba

Carla,22,Fortaleza
```

Cada linha representa um registro, e cada valor separado por vírgula é um campo do registro.

A função *fscanf()* é semelhante à função *scanf()*, mas em vez de ler dados do teclado, ela lê dados de um arquivo. Ela permite especificar o formato dos dados a serem lidos e armazená-los em variáveis apropriadas.

Código completo em C da leitura do arquivo “dados.csv”:

```
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    char nome[50];
    int idade;
    char cidade[50];
} Pessoa;

int main() {
    FILE *arquivo;
    Pessoa pessoa;

    // Abre o arquivo CSV para leitura
    arquivo = fopen("dados.csv", "r");
    if (arquivo == NULL) {
        perror("Erro ao abrir o arquivo");
        return 1;
    }

    printf("Leitura dos dados do arquivo:\n");
    printf("————————————————————————————————————\n");

    // Lê linha por linha usando fscanf
    while (fscanf(arquivo, "%49[^,],%d,%49[^\n]\n",
                  pessoa.nome,
                  &pessoa.idade,
                  pessoa.cidade) == 3) {
        printf("Nome: %-10s | Idade: %2d | Cidade: %s\n",
               pessoa.nome, pessoa.idade, pessoa.cidade);
    }
    fclose(arquivo);
    return 0;
}
```

O programa abre o arquivo “dados.csv” para leitura e utiliza *fscanf()* para ler cada linha e extrair os valores separados por vírgula. Formatação de leitura dos campos:

- ➔ **%49[^,]** : lê até encontrar uma vírgula (usado para o nome e a cidade);
- ➔ **%d** : lê um número inteiro (a idade);
- ➔ **%49[^\n]** : lê até o final da linha.

Estrutura de dados STL

Para esta atividade, é fundamental compreender e aplicar as estruturas de dados da *Standard Template Library* (STL) do C++:

std::vector: estrutura dinâmica que armazena uma sequência de elementos contíguos na memória, mas cujo tamanho pode variar durante a execução do programa. Seu acesso a qualquer elemento é feito em tempo constante.

Código exemplo que utiliza o vector:

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    // Criação de um vetor dinâmico de inteiros
    vector<int> numeros;

    // Inserção de elementos
    numeros.push_back(10);
    numeros.push_back(20);
    numeros.push_back(30);

    // Acesso direto por índice
    cout << "Segundo elemento: " << numeros[1] << endl;  // Saída: 20

    // Inserindo mais elementos dinamicamente
    numeros.push_back(40);
    numeros.push_back(50);

    cout << "Tamanho atual do vetor: " << numeros.size() << endl;

    return 0;
}
```

Principais funções do vector no código:

- ➔ **vector<int> numeros** : cria um vetor dinâmico vazio de inteiros.
- ➔ **push_back(valor)** : insere o valor no final do vetor.
- ➔ **numeros[i]** : acessa o elemento na posição *i*.
- ➔ **size()** : retorna a quantidade atual de elementos armazenados.

std::list: Uma lista encadeada é uma estrutura de dados linear, formada por uma sequência de elementos (nós), onde cada nó armazena um valor e um ponteiro para o próximo nó da sequência. Diferentemente de vetores, os elementos da lista não estão armazenados em posições contíguas na memória; eles são ligados entre si por referências (ponteiros). Sendo assim, a std::list não suporta acesso direto via índice (como lista[2]).

Código exemplo que utiliza o list:

```
#include <iostream>
#include <list>
using namespace std;

int main() {
    // Criação de uma lista de inteiros
    list<int> numeros;

    // Inserindo elementos no final da lista
    numeros.push_back(10);
    numeros.push_back(20);
    numeros.push_back(30);

    // Exibindo todos os elementos (iteração)
    cout << "Conteúdo da lista: ";
    for (int valor : numeros) {
        cout << valor << " ";
    }
    cout << endl;

    // Limpando a lista
    numeros.clear();
    cout << "Tamanho da lista após clear(): " << numeros.size() << endl;

    return 0;
}
```

Principais funções do vector no código:

- ➔ **list<int> numeros** : cria uma lista de inteiros vazia.
- ➔ **push_back(valor)** : insere um novo elemento no final da lista.
- ➔ **for (int valor : numeros)** : percorre a lista elemento por elemento (não há índices).
- ➔ **clear()** : remove todos os elementos da lista.

<u>std::map</u>: Um mapa, também conhecido como dicionário ou tabela de associação, é uma estrutura de dados que armazena pares de chave e valor (**key → value**). Cada chave é única dentro do mapa, e é usada para acessar o valor associado de forma eficiente. Não permite inserção de chaves duplicadas.

Código exemplo que utiliza o map:

```
#include <iostream>
#include <map>
#include <string>
using namespace std;

int main() {
    // Criação de um mapa que associa nomes (chave) a idades (valor)
    map<string, int> idade;

    // Inserindo elementos no mapa
    idade["Ana"] = 25;
    idade["Bruno"] = 30;
    idade["Carla"] = 22;

    // Exibindo todos os pares (ordem crescente das chaves)
    for (auto& p : idade) {
        cout << p.first << " → " << p.second << endl;
    }

    // Pesquisando uma chave
    string nome = "Bruno";
    if (idade.find(nome) != idade.end()) {
        cout << nome << " está no mapa, idade = " << idade[nome] << endl;
    } else {
        cout << nome << " não encontrado." << endl;
    }

    // Verificando antes de inserir (para evitar sobrescrita)
    string novo_nome = "Ana";
    int nova_idade = 28;

    if (idade.find(novo_nome) == idade.end()) {
        idade[novo_nome] = nova_idade;
        cout << "Inserido: " << novo_nome << " → " << nova_idade << endl;
    } else
        cout << "Aviso: " << novo_nome << " já está no mapa, não inserido novamente.\n";
     return 0;
}
```

Principais funções do vector no código:

- ➔ **map<string, int> idade** : cria um mapa cujas chaves são strings (nomes) e os valores são int (idades).
- ➔ **idade["Ana"] = 25** : cria o par "Ana" → 25 se a chave ainda não existir. Sobrescreve o valor se a chave já existir.
- ➔ **find()** : retorna um iterador para o elemento, ou *end()* se a chave não for encontrada.

Entregáveis:

Os alunos deverão entregar o código-fonte do módulo <u>ListaCompras</u> e um programa testador. O testador deve ser capaz de exibir os nomes dos produtos comprados por pelo menos 3 clientes, utilizando seus códigos originais como entrada.

Atividade 2: Módulo Similaridade

Descrição:

Implementação do módulo Similaridade, que calcula e fornece o valor da função de similaridade entre dois clientes, conforme a definição da distância de Jaccard apresentada na Seção 2.2. A identificação dos clientes para este módulo deve ser interna (índices), não os códigos originais da base de compras. Nesta primeira versão, o módulo Similaridade receberá os dados do módulo ListaCompras, transformará esses dados em uma matriz de compras densa e, em seguida, computará a matriz de similaridade multiplicando a matriz de compras por sua transposta.

Teoria aplicada:

Algoritmo de Multiplicação de Matrizes Padrão:

A multiplicação de matrizes é uma operação que combina duas matrizes A e B para produzir uma terceira matriz C. Seja A de dimensão $m \times n$ e B de dimensão $n \times p$, então o produto $C = A \times B$ é uma matriz de dimensão $m \times p$.

Na multiplicação de matriz cada célula $C[i][j]$é computada da seguinte forma:

$$C[i][j] = \sum_{k=1}^{n} A[i][k] \times B[k][j]$$

Pseudocódigo da multiplicação de matrizes.

```
Algoritmo MultiplicaMatrizes(A, B)
  Entrada:
    A[m][n] - matriz de dimensão m x n
    B[n][p] - matriz de dimensão n x p
  Saída:
    C[m][p] - matriz resultado do produto A x B

  Para i ← 1 até m faça
      Para j ← 1 até p faça
          C[i][j] ← 0
          Para k ← 1 até n faça
              C[i][j] ← C[i][j] + A[i][k] * B[k][j]
          FimPara
      FimPara
  FimPara

  Retorne C
FimAlgoritmo
```

Matriz Transposta:

A matriz transposta de uma matriz A, denotada por A^T, é obtida trocando suas linhas por colunas. Formalmente:

$$A^T[i][j] = A[j][i].$$

Ou seja, a primeira linha de A vira a primeira coluna de A^T; a segunda linha vira a segunda coluna, e assim por diante.

Pseudocódigo da transposta da matriz:

```
Algoritmo Transposta(A)
  Entrada:
    A[m][n] – matriz de dimensão m × n
  Saída:
    T[n][m] – matriz transposta de A

  Para i ← 1 até m faça
      Para j ← 1 até n faça
          T[j][i] ← A[i][j]
      FimPara
  FimPara

  Retorne T
FimAlgoritmo
```

Entregáveis:

Os alunos deverão entregar o código-fonte do módulo Similaridade e um programa testador. O testador deve ser capaz de, para dois clientes quaisquer (identificados por seus índices internos), encontrar e exibir o cliente mais similar que não seja ele mesmo.

Atividade 3: Módulo Recomendação

Descrição:

O módulo Recomendação será responsável por gerar as recomendações de produtos para cada cliente da loja, utilizando o algoritmo de ranqueamento descrito na Seção 2.3 deste projeto. O módulo deverá retornar os *k* produtos mais bem ranqueados para um determinado cliente.

O vetor de ranqueamento de cada cliente deve ser composto por uma estrutura contendo o identificador interno do produto e o respectivo valor de ranqueamento. Esse vetor deve ser ordenado por meio de um algoritmo de ordenação, tomando como base o valor de ranqueamento de cada produto.

Teoria aplicada:

Algoritmos de Ordenação

Os algoritmos de ordenação são métodos utilizados para reorganizar elementos de uma coleção (como vetores, listas ou registros) segundo uma determinada ordem, normalmente crescente ou decrescente, com base em algum critério de comparação.

Ordenar dados é uma das tarefas mais importantes em computação, pois:

- Melhora a eficiência de busca (por exemplo, em uma busca binária);
- Facilita a análise de dados e a visualização de padrões.

Código exemplo em C para ordenar um vetor registros de uma estrutura personalizada:

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <string>

struct Produto {
    std::string nome;
    double preco;
};

// Função de comparação: retorna true se 'a' deve vir antes de 'b'
bool compararPorPreco(const Produto &a, const Produto &b) {
    return a.preco < b.preco; // ordem crescente de preço
}

int main() {
    std::vector<Produto> produtos = {
        {"Teclado", 150.0},
        {"Mouse", 80.0},
        {"Monitor", 900.0}
    };

    // Ordena o vetor usando a função de comparação
    std::sort(produtos.begin(), produtos.end(), compararPorPreco);

    std::cout << "Produtos ordenados por preço:\n";
    for (const auto &p : produtos)
        std::cout << p.nome << " - R$ " << p.preco << std::endl;

    return 0;
}
```

Entregáveis:

Os alunos deverão entregar o código-fonte do módulo Recomendação e um programa testador. O testador deve solicitar a recomendação para 3 clientes, utilizando seus códigos originais, e exibir os nomes dos *k* produtos melhor recomendados para cada cliente, demonstrando a funcionalidade completa do sistema de recomendação.